Լաբորատոր աշխատանք 8

Դասերի օգտագործում և կոնստրուկտորների գերբեռնում

Սարքավորման վարձակալություն

Աշխատանքի նպատակը. Ուսումնասիրել C++ լեզվում դասերի օգտագործումը, կոնստրուկտորների գերբեռնումը, տվյալների ստուգումը և օբյեկտների զանգվածների մշակումը։

Խնդրի պահանջը. Ստեղծել EquipmentRental դաս՝ սարքի անվան, վարձակալության օրերի, օրական գնի և զեղչի տոկոսով։ Իրականացնել երեք կոնստրուկտոր, հաշվարկային և ցուցադրող մեթոդներ, ինչպես նաև վարձակալության ժամկետը երկարացնող մեթոդ։ Ստեղծել երեք օբյեկտ և հինգ օբյեկտից բաղկացած զանգված, ստուգել մուտքագրվող տվյալները, գտնել ամենաթանկ վարձակալությունը և հաշվել վերջնական արժեքների ընդհանուր գումարը։

Առաջադրանքը. Ստեղծել EquipmentRental դասը։ Պահպանվող տվյալներն են՝ սարքի անունը, վարձակալության օրերի քանակը, օրական գինը և զեղչի տոկոսը։

1. Կոնստրուկտորներ։ EquipmentRental()՝ ընդհանուր կանոններով, երեք պարամետրով՝ անուն, օրերի քանակ, օրական գին (զեղչը՝ 0), չորս պարամետրով՝ բոլոր տվյալները։

2. Ստուգումներ։ օրերի քանակը և գինը՝ դրական, զեղչը՝ 0–20։ Քանակները և օրերը ամբողջ թվեր են, չափումները, գներն ու տոկոսները՝ double։

3. Փոփոխող մեթոդ։ Երկարացնել վարձակալությունը դրական թվով օրերով։

4. Հաշվարկային մեթոդներ։ Հաշվել հիմնական և զեղչված արժեքները։

5. Զանգվածի մշակում։ Ցուցադրել ամենաթանկ վարձակալությունը և վերջնական արժեքների գումարը։ Որոնման ժամանակ օգտագործել համապատասխան հաշվարկային մեթոդի արդյունքը։

6. Ստուգիչ օրինակ։ Մուտք՝ օրեր՝ 5, օրական գին՝ 4000, զեղչ՝ 15%։ Ակնկալվող արդյունք՝ հիմնական արժեքը՝ 20000, վերջնականը՝ 17000։ Օրինակը վերաբերում է օբյեկտի սկզբնական վիճակին՝ մինչև փոփոխող մեթոդի կանչը։

```
#include <iostream>
#include <string>
```

```
#include <limits>
#include <iomanip>
#include <cmath>
using namespace std;
class EquipmentRental
{
private:
    string equipmentName;
    int days;
    double dailyPrice;
    double discount;
public:
    // Constructor without parameters
    EquipmentRental()
    {
        equipmentName = "Unknown";
        days = 1;
        dailyPrice = 1;
        discount = 0;
    }
    // Constructor with three parameters
    EquipmentRental(string name, int d, double price)
    {
        equipmentName = name.empty() ? "Unknown" : name;
        if (d > 0)
            days = d;
        else
        {
            days = 1;
            cout << "Invalid days! Default value 1 is used.\n";
```

```
    }
    if (isfinite(price) && price > 0)
        dailyPrice = price;
    else
    {
        dailyPrice = 1;
        cout << "Invalid price! Default value 1 is used.\n";
    }
    discount = 0;
}
// Constructor with four parameters
EquipmentRental(string name, int d, double price, double disc)
{
    equipmentName = name.empty() ? "Unknown" : name;
    if (d > 0)
        days = d;
    else
    {
        days = 1;
        cout << "Invalid days! Default value 1 is used.\n";
    }
    if (isfinite(price) && price > 0)
        dailyPrice = price;
    else
    {
        dailyPrice = 1;
        cout << "Invalid price! Default value 1 is used.\n";
    }
    if (isfinite(disc) && disc >= 0 && disc <= 20)
        discount = disc;
```

```
    else
    {
        discount = 0;
        cout << "Invalid discount! Default value 0 is used.\n";
    }
}
// Calculates the original rental cost
double calculateBaseCost() const
{
    return days * dailyPrice;
}
// Calculates the rental cost after discount
double calculateFinalCost() const
{
    return calculateBaseCost() * (1 - discount / 100.0);
}
// Displays all data and calculated costs
void show() const
{
    cout << "Equipment name: " << equipmentName << endl;
    cout << "Rental days: " << days << endl;
    cout << "Daily price: " << dailyPrice << endl;
    cout << "Discount: " << discount << "%" << endl;
    cout << "Base cost: " << calculateBaseCost() << endl;
    cout << "Final cost: " << calculateFinalCost() << endl;
}
// Extends the rental period
void extendRental(int extraDays)
{
    if (extraDays > 0 &&
```

```
            days <= numeric_limits<int>::max() - extraDays)
        {
            days += extraDays;
            cout << "Rental period extended successfully.\n";
        }
        else
        {
            cout << "Invalid number of days! Rental unchanged.\n";
        }
    }
};
// Reads a non-empty equipment name
string inputString(string message)
{
    string value;
    while (true)
    {
        cout << message;
        getline(cin, value);
        if (value.find_first_not_of(" \t\r\n") != string::npos)
            return value;
        cout << "Name cannot be empty! Try again.\n";
    }
}
// Reads a positive integer
int inputPositiveInt(string message)
{
    int value;
    while (true)
    {
```

```
        cout << message;
        if (cin >> value && value > 0)
        {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Invalid input! Enter a positive integer.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}
// Reads a positive finite number
double inputPositiveDouble(string message)
{
    double value;
    while (true)
    {
        cout << message;
        if (cin >> value && isfinite(value) && value > 0)
        {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Invalid input! Enter a positive number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}
// Reads a discount from 0 to 20
double inputDiscount(string message)
```

```
{
    double value;
    while (true)
    {
        cout << message;
        if (cin >> value && isfinite(value) &&
            value >= 0 && value <= 20)
        {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Invalid discount! Enter a value from 0 to 20.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}
int main()
{
    cout << fixed << setprecision(2);
    // Creates one object using each constructor
    cout << "========== THREE CONSTRUCTORS ==========\n";
    EquipmentRental rental1;
    EquipmentRental rental2(
        inputString("Enter equipment name for object 2: "),
        inputPositiveInt("Enter rental days: "),
        inputPositiveDouble("Enter daily price: ")
    );
    EquipmentRental rental3(
        inputString("Enter equipment name for object 3: "),
        inputPositiveInt("Enter rental days: "),
```

```
    inputPositiveDouble("Enter daily price: "),
    inputDiscount("Enter discount (0-20): ")
);
cout << "\n========== OBJECT 1 ==========\n";
rental1.show();
cout << "\n========== OBJECT 2 ==========\n";
rental2.show();
cout << "\n========== OBJECT 3 ==========\n";
rental3.show();
// Creates an array of five objects
EquipmentRental rentals[5];
cout << "\n========== ENTER 5 RENTALS ==========\n";
// Reads valid data and creates each object with four parameters
for (int i = 0; i < 5; i++)
{
    cout << "\nRental " << i + 1 << ":\n";
    string name = inputString("Equipment name: ");
    int d = inputPositiveInt("Rental days: ");
    double price = inputPositiveDouble("Daily price: ");
    double disc = inputDiscount("Discount (0-20): ")
    rentals[i] = EquipmentRental(name, d, price, disc);
}
// Displays all rentals before modification
cout << "\n========== INITIAL RENTAL DATA ==========\n";
for (int i = 0; i < 5; i++)
{
    cout << "\nRental " << i + 1 << ":\n";
    rentals[i].show();
}
// Selects one object to extend its rental period
```

```
int selected;
while (true)
{
    selected = inputPositiveInt(
        "\nSelect a rental to extend (1-5): "
    );
    if (selected >= 1 && selected <= 5)
        break;
    cout << "Invalid selection! Choose a number from 1 to 5.\n";
}
int extraDays = inputPositiveInt("Enter additional days: ");
rentals[selected - 1].extendRental(extraDays);
cout << "\n========== UPDATED RENTAL ==========\n";
rentals[selected - 1].show();
// Finds the highest final cost and calculates the total
int maxIndex = 0;
double total = 0;
for (int i = 0; i < 5; i++){
    double finalCost = rentals[i].calculateFinalCost();
    total += finalCost;
    // Strict comparison preserves the first matching maximum
    if (finalCost > rentals[maxIndex].calculateFinalCost()){
        maxIndex = i;}}
// Displays the most expensive rental after discount
cout << "\n========== MOST EXPENSIVE RENTAL ==========\n";
rentals[maxIndex].show();
// Displays the sum of all final rental costs
cout << "\n========== TOTAL FINAL COST ==========\n";
cout << "Total: " << total << endl;
return 0;}
```